

Problem Set: Triaging submissions to an ontology registry with Claude

Chapter 1 — Introduction

Topic coverage: Keet §1.1 (what an ontology looks like: the vocabulary → taxonomy → thesaurus → formal-ontology spectrum), §1.2 (why ontologies pay off), §1.3 (what makes an ontology good or bad); course notebooks 01–04 of this chapter. Tools: LangChain tool-calling agents, DSPy + GEPA, an LLM judge, `oe_course.mdp`, `oe_course.skills`, `oe_course.selfimprove`. This is the course's foundation problem set: every later chapter reuses its discipline.

Level: Graduate (practice-first). Combines conceptual analysis with building, measuring and optimising a real Claude-backed agent.

Total points: 100 (Part A 18 / Part B 32 / Part C 26 / Part D 24); 53 pts are auto-graded by the check cells, the rest are marked by hand against the rubric. **Estimated effort:** 10–12 hours.

API budget: ≈ \$8–20 **estimated** on `claude-opus-5` for one full run (≈ 330 DSPy calls and ≈ 60 agent turns; the two GEPA runs and the noise study are the largest lines). Nothing in this notebook has been run live by the authors, so these are estimates from list prices (5/25 per MTok in / out), not measurements — your `llm.meter` lines are the real numbers. Re-running unchanged cells is free: DSPy caches identical requests. Every model call in this notebook is a billed request to Anthropic; the cost lines the problems ask for are part of the answer.

The brief

Meridian Rail runs a company-wide **ontology registry**: before a team's vocabulary or ontology is published for other systems to depend on, the data architecture office reviews it at *intake*. The review answers three questions — where does the submission sit on the ontology spectrum, which modelling defects does it have, and is it **accepted**, sent back to **revise**, or **rejected** — and ends with a short report to the submitting team. The rules are the office's written *intake policy (v3)*, printed below. The backlog is 24 submissions from 16 teams and growing; the head of data architecture wants to know whether Claude can take over first-line triage, and on what evidence. You are the ontology engineer asked to find out. Your brief:

1. **Pin the task down.** Write the policy's decidable clauses as code, and establish what a deterministic reviewer with the registry's tools already achieves. That is the control every Claude number is compared with.

2. **Build the grader first**, with diagnostics an optimiser can act on — and validate the LLM judge the office wants to use for the written reports *before* trusting it.
3. **Build a Claude triage program**, measure it on a development split, improve it with GEPA, and report on a **held-out test split** with its cost and its run-to-run noise.
4. **Decide the architecture**: a single tool-calling agent, or a decomposed pipeline with a verifying critic; price evidence-gathering as an MDP; and ship the result as a versioned skill behind a gated self-improvement loop.

The 24 submissions (split 8 / 8 / 8 by submission, each split containing every decision and every spectrum level), the board's gold labels, the registry tools and a set of reviewer-labelled reports are provided in `ch01_agentic.py`. The gold labels are checked against Chapter 1's own spectrum classifier and defect scanner when the notebook starts.

Submission requirements

1. **This notebook, executed top to bottom**, with every `TODO` implemented and every *Your answer* cell written.
2. **The artefacts it saves** under `course/artifacts/` (optimised instructions, run logs) — they are evidence, not by-products.
3. **Honest reporting**. A number without its split, its sample size and its cost earns no credit. Report the runs you did, including the ones that did not improve anything.

The check cells are the marking scheme for the auto-graded points; do not edit them. The final self-test prints your auto-graded subtotal.

Solutions: `05_solutions.ipynb` — do not open it until you have attempted every problem.

```
In [ ]: import sys, json, logging
        from pathlib import Path

        # Make the course root and this chapter importable no matter where Jupyter started.
        here = Path.cwd()
        for candidate in [here, *here.parents]:
            if (candidate / "oe_course").is_dir():
                sys.path.insert(0, str(candidate))
                break
        sys.path.insert(0, str(here))

        import oe_course
        from oe_course import config
        from oe_course.grading import Grader

        print(json.dumps(oe_course.describe_environment(), indent=1))
        config.require_credentials() # this problem set calls the Live Anthropic API
        logging.getLogger("dspy").setLevel(logging.WARNING)
        GRADER = Grader('Chapter 1 – Introduction – Triage submissions to an ontology reg
```

```

import re, time
import statistics as st
from typing import Iterable
import dspy
import pandas as pd
import ch01_agentic as A
from oe_course import evaluation as ev, llm, mdp, optimize as opt, agents, ontology
from oe_course.skills import Skill
from oe_course.selfimprove import SelfImprovingSkill
pd.set_option("display.width", 140)
pd.set_option("display.max_colwidth", 70)

problems = A.validate_gold()
assert not problems, problems      # the gold labels agree with Chapter 1's engine
train, dev, test = A.build_dataset("train"), A.build_dataset("dev"), A.build_dataset("test")
ALL = train + dev + test
ROW = {ex.id: ex for ex in ALL}
print(f"train {len(train)} / dev {len(dev)} / test {len(test)}; gold labels validated")

```

The corpus and the policy

One row per submission. `level` and `raw_smells` are what Chapter 1's classifier and scanner report (validated above); `smells` and `decision` are the review board's gold answer under the policy.

```

In [ ]: print(A.INTAKE_POLICY)
pd.DataFrame([{"id": ex.id, "split": ex.split, "team": ex.team, "declared": ex.declared,
               "level": ex.level, "raw_smells": ex.raw_smells, "smells": ex.smells,
               "decision": ex.decision} for ex in ALL])

```

Where the submissions live, and how to load them

There are no separate `.ttl` files to download. The 24 submissions are **embedded as Turtle text** in `ch01_agentic.py` (the list `A.SUBMISSIONS`), so the problem set runs with no file paths and no triplestore. Each entry is an `A.Submission`; `A.get(id)` returns one, and its `.turtle` attribute is a complete Turtle document (prefix header included).

Three ways to load one, from lowest to highest level:

you want	call	you get
the raw text	<code>A.get("safety-incidents").turtle</code>	<code>str</code> (Turtle)
an RDF graph to inspect in Python	<code>ont.load_graph(A.get(id).turtle) (= rdflib.Graph().parse(data=..., format="turtle"))</code>	<code>rdflib.Graph</code>
everything the tools need	<code>ws = A.RegistryWorkspace(); ws.load(id)</code>	<code>ws.graph</code> (rdflib) and <code>ws.store</code> (a SPARQL store over the same triples)

The tools never read files: `load_submission(id)` calls `ws.load(id)`, and every other tool works on `ws.graph` / `ws.store`. Fuseki is not needed; if it is running, nothing in this problem set uses it.

The next cell prints one submission, loads it three ways, and **exports all 24 as .ttl files** under `course/artifacts/ch01_submissions/`, so you can open them in Protégé or any RDF editor. Editing those files does not change the corpus; the module is the source of truth.

```
In [ ]: from rdflib import Graph

sub = A.get("safety-incidents")
print(sub.turtle)

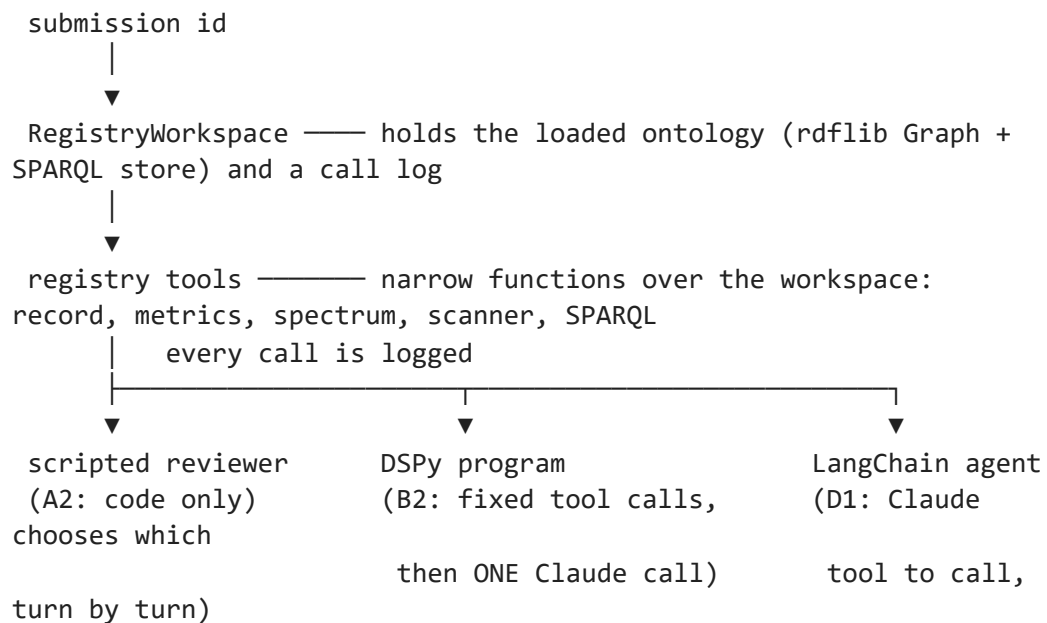
g = ont.load_graph(sub.turtle)                                # rdflib.Graph
print(f"{len(g)} triples;", ont.classify_spectrum(g)["level"], ";",
      sorted(ont.smell_summary(g)))

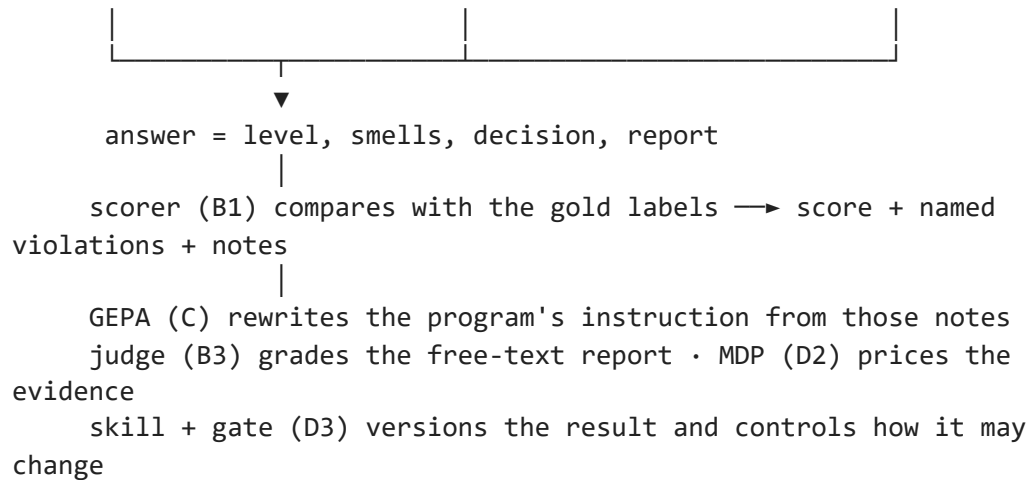
ws_src = A.RegistryWorkspace()
ws_src.load(sub.id)                                           # what Load_submission does
print(ws_src.store.select("SELECT ?c WHERE { ?c a owl:Class } ORDER BY ?c")[:3])

out_dir = config.artifacts_dir() / "ch01_submissions"
out_dir.mkdir(parents=True, exist_ok=True)
for s in A.SUBMISSIONS:
    (out_dir / f"{s.id}.ttl").write_text(s.turtle, encoding="utf-8")
print(f"\nexported {len(A.SUBMISSIONS)} files to {out_dir}")
```

Background — the system you are building

This section explains the moving parts and **why** they are shaped the way they are. The problems ask you to make design decisions; these are the concepts those decisions are about.





1. The task: intake triage

A team submits an ontology; the registry answers four things: its **level** on the ontology spectrum, the **defects** to report, a **decision**, and a **report** to the team. The first three follow mechanically from the policy and the tool output; the fourth is language. That split is the reason the problem set is built the way it is: it lets you measure exactly how faithfully Claude applies written rules, and shows where a model adds something code cannot.

The **spectrum classifier** (`ont.classify_spectrum`) is cumulative: any subsumption (or `skos:broader`) makes a *taxonomy*; SKOS relations (`broader` , `related` , ...) make a *thesaurus*; restrictions, disjointness, equivalence or property characteristics **on top of a hierarchy** make a *formal-ontology*. The **scanner** (`ont.scan_smells`) returns one finding per defect instance.

2. The workspace — A.RegistryWorkspace

A workspace is the **state one review works on**: which submission is loaded (`ws.submission`), its RDF graph (`ws.graph`), a SPARQL store over it (`ws.store`), and the log of every tool call made against it (`ws.log`). Think of it as a session or a sandbox.

Why an object instead of global variables:

- **Isolation.** The tools take almost no arguments (`spectrum_position()` means "of the loaded submission"), so they need somewhere to find that submission. If that were a global, two reviews running at once — or an agent run and your own debugging — would overwrite each other. One workspace per review keeps them apart.
- **A record per episode.** The log belongs to the workspace, so one workspace = one trajectory = one cost. That is what makes A2's comparison, D1's tool counts and D2's replay possible.
- **Reproducibility.** A fresh workspace always starts empty; nothing leaks from the previous item.

Design decisions: a **fresh workspace per item** when you want per-item trajectories (A2, D1); a **shared workspace** when you want one log for a whole run, e.g. to cost a dataset evaluation (`A.gather_evidence(id, ws)` accepts one for this reason).

3. Tools and the tool registry — `A.build_registry_tools(ws)`

A **tool** is a Python function wrapped as a LangChain `StructuredTool`: a `name`, a `description` (the docstring) and an argument schema inferred from the type hints. The same tool object can be used in three ways:

1. **by your code:** `tool.invoke({"submission_id": "x"})`, a normal function call (A2, B2);
2. **by Claude:** the agent receives each tool's name, description and schema, and *decides* which to call and with what arguments (D1);
3. **inside a fixed pipeline:** `A.gather_evidence` calls five tools in a fixed order.

The **tool registry** (or *tool belt*) is simply the list `A.build_registry_tools(ws)` returns: the actions available to a reviewer, all bound to one workspace. Choosing that list is a design decision in its own right.

Design choices already made in these tools, and why:

- **Narrow tools** (one measurement each) rather than one `review_everything` tool: the trajectory shows *what* evidence a decision used; an agent can be given only what it needs (least privilege); outputs stay small, and every agent turn re-sends them. `A.build_monolithic_tool` exists so A2 can measure the difference.
- **Descriptions say when to call**, not only what the tool does ("Call this before deciding, because ..."). A model picks tools from their descriptions; a missing trigger sentence means a tool that never gets called.
- **JSON strings in, JSON strings out:** readable by a model, loggable, and identical whether a person or a model made the call.
- **Errors come back as text** (`"ERROR: No submission loaded..."`) instead of exceptions, so an agent can read the mistake and recover instead of crashing the run.

4. The call log — `ws.log`

Every tool is wrapped by `oe_course.tools.instrument`, which records a `ToolCall` (name, arguments, success, seconds, size of the result) before returning. The log **is** the agent's trajectory. You read it to cost a run, to see which evidence a decision rested on (A2), to count tool calls (D1) and to replay an agent's episode inside the MDP (D2).

5. Evidence bundle, and "tools measure, the model interprets"

`A.gather_evidence(id)` runs the record, metrics, spectrum and scanner tools and returns their outputs as one JSON document. The B2 program gives that document to Claude in a **single** call. The design principle: put measurement in code (exact, free, repeatable) and use the model only for interpretation and language. It makes the system cheaper and easier to evaluate, and it leaves exactly one instruction to optimise.

6. DSPy — signature, predictor, module

- A **signature** (`class X(dspy.Signature)`) is the typed contract of one LM call: input fields, output fields, their descriptions, and an **instruction** (the docstring, or `.with_instructions(...)`). It replaces a hand-written prompt string.
- A **predictor** (`dspy.Predict(signature)`) turns the signature into a prompt, makes one LM call and parses the reply back into typed fields.
- A **module** (`dspy.Module` with a `forward` method) is your program: ordinary Python that calls tools and predictors. Calling it returns a `dspy.Prediction` .

The point of DSPy here: the **instruction is a parameter**, separate from the code, so an optimiser can rewrite it and you can version, diff and review it like any other artefact.

7. Scorer, feedback and rulebook

A **scorer** turns (gold row, prediction) into a `ScoreReport` : a number, a list of **named violations** (ids from `A.INTAKE_RULEBOOK`) and **notes** that say what was expected. The number drives evaluation; the ids and notes drive optimisation, because GEPA can only fix what the feedback names. A scorer that returns only a number gives an optimiser nothing to act on.

8. GEPA

GEPA improves an instruction by **reflection**: it runs the program on a few training items, collects the metric's feedback text, asks a *reflection LM* to write a better instruction, and keeps candidates that do best on the validation set. Its budget (`max_metric_calls`) is the number of program runs it may make; each is a billed call. It learns only from failures that occur in `train` , which is why the splits matter as much as the optimiser.

9. LLM-as-judge

A judge is a model that scores something no function can, here the written report. A judge is itself a classifier with errors, so it must be **validated against human labels** (agreement, κ) before its scores are trusted. B3 does this before anything relies on it.

10. Agent loop vs pipeline

An **agent** (D1) is a loop: Claude sees the conversation and the tool schemas, either requests a tool call or answers; each tool result is appended and the loop repeats (up to 25 steps here). It is flexible, since the order of work can depend on what it finds, but each turn re-sends the whole transcript, the trajectory varies from run to run, and it can skip evidence. A **pipeline** fixes the order in code; a **critic** is a pipeline stage that checks the model's claims against raw tool output. Use an agent only when the order of work genuinely depends on intermediate results.

11. MDP

Formalising the review as a Markov decision process (states = evidence held, actions = tool calls or *submit*, reward = answer quality minus a cost per call) turns "is the agent efficient?" into arithmetic: solve for the optimal value V^* , and an agent's **regret** is how far below V^* its actual trajectory scored.

12. Skill and gated self-improvement

A **skill** bundles an instruction with the tools it assumes, the data it is measured on and the metric, and versions them together, so a change in the prompt always travels with the evidence for it. **Self-improvement** means mining the skill's own failures, re-optimising, and **promoting a new version only if it beats the current one by a margin on a holdout the optimiser never saw**. Without that gate, "self-improving" means drifting while confidently reporting progress.

Reference — data, types and APIs used in this problem set

Read this once before Part A, and come back to it whenever a contract below names a type. Every problem also has its own **Contract** block, just before its TODO cell, with the exact signatures, what each parameter is, what must be returned, and which variables the check cell reads.

Type notation

`list[str]` a Python list of strings · `frozenset[str]` an immutable set · `dict[str, bool]` a dict with string keys and bool values · `Iterable[str]` anything you can loop over (list, set, tuple, `pd.Series`) · `T | None` a `T` or `None` · `pd.DataFrame` a pandas table (column names given in each contract).

Dataset rows — `train`, `dev`, `test`, `ALL`, `ROW`

`train`, `dev` and `test` are `list[dspy.Example]` (8 each); `ALL` = `train` + `dev` + `test` (24); `ROW` is `dict[str, dspy.Example]` keyed by submission id. Read fields as

attributes (`ex.level`) or keys (`ex["level"]`). **The input** is `submission` — that is what `program(**ex.inputs())` passes; everything else is a gold label or metadata.

field	type	example	meaning
<code>id</code> , <code>submission</code>	<code>str</code>	<code>"safety-incidents"</code>	the submission id (same value; <code>submission</code> is the program input)
<code>split</code>	<code>str</code>	<code>"train"</code>	<code>"train"</code> , <code>"dev"</code> or <code>"test"</code>
<code>team</code>	<code>str</code>	<code>"Safety & Incidents"</code>	submitting team
<code>declared_level</code>	<code>str</code>	<code>"formal-ontology"</code>	what the team <i>claims</i> on the intake form
<code>level</code>	<code>str</code>	<code>"formal-ontology"</code>	gold: what the spectrum classifier <i>measures</i>
<code>raw_smells</code>	<code>list[str]</code> , sorted	<code>["missing-label", "property-without-domain-or-range"]</code>	gold: every smell id the scanner finds
<code>smells</code>	<code>list[str]</code> , sorted	same, minus waived ids	gold: what the policy says must be <i>reported</i>
<code>decision</code>	<code>str</code>	<code>"revise"</code>	gold: <code>"accept"</code> , <code>"revise"</code> or <code>"reject"</code>
<code>note</code>	<code>str</code>		why the board labelled it so (do not use it as model input)

Constants

name	value
<code>ont.SPECTRUM_LEVELS</code>	<code>["controlled-vocabulary", "taxonomy", "thesaurus", "formal-ontology"]</code> — ordered from least to most formal
<code>A.DECISIONS</code>	<code>["accept", "revise", "reject"]</code>
<code>A.BLOCKING_SMELLS</code>	<code>frozenset({"subsumption-cycle", "class-as-individual", "individual-as-class", "undeclared-term"})</code>
<code>A.WAIVABLE_AT</code>	<code>frozenset({"controlled-vocabulary", "thesaurus"})</code> — measured levels at which <code>no-disjointness</code> is waived (W1)
<code>A.EVIDENCE_KINDS</code>	<code>["record", "metrics", "spectrum", "smells", "catalogue", "sparql"]</code>
<code>A.TOOL_TO_EVIDENCE</code>	<code>dict[str, str] : tool name → evidence kind ("scan_smells" → "smells" , ...); tools not listed buy nothing</code>

name	value
A.INTAKE_RULEBOOK	RuleBook of 7 Rule(id, description) ; A.INTAKE_RULEBOOK.ids is the ordered id list; "x" in A.INTAKE_RULEBOOK

The seven scanner smell ids (ont.SMELL_IDS): subsumption-cycle , class-as-individual , individual-as-class , undeclared-term (these four are blocking), property-without-domain-or-range , no-disjointness , missing-label .

Registry tools — A.build_registry_tools(ws)

ws = A.RegistryWorkspace() is a fresh, empty workspace with its own call log ws.log .
A.build_registry_tools(ws) returns a list of LangChain tools bound to ws ; index them with tools = {t.name: t for t in A.build_registry_tools(ws)} . Call a tool with tools[name].invoke({arg: value}) ({} for no arguments). Every tool returns a **JSON string** — decode it with json.loads . A tool that fails returns a string starting with "ERROR:" and the call is logged with ok=False . Call load_submission first.

tool	arguments	returns (after json.loads)
load_submission	submission_id: str	{"loaded": "safety-incidents", "triples": 15}
submission_record	—	{"submission": ..., "team": ..., "declared_level": "formal-ontology", "purpose": ...}
graph_metrics	—	{"classes": 5, "subclass_axioms": 3, "disjointness_axioms": 1, "labels": 5, ...} (17 counts)
spectrum_position	—	{"level": "formal-ontology", "evidence": ["5 classes, 5 labels -> ...", ...]}
scan_smells	—	[{"smell": "missing-label", "subject": "<IRI>", "detail": "..."}, ...] — one entry per finding (an id can repeat)
smell_catalogue	—	[{"id": ..., "title": ..., "why": ...}, ...]
sparql_select	query: str	list[dict] (≤ 100 rows)
sparql_ask	query: str	{"answer": true}
list_submissions	—	list[str] of ids

A.build_monolithic_tool(ws) returns a one-element list holding
review_submission(submission_id) , which returns everything at once.

The call log. `ws.log` is a `ToolCallLog`; `ws.log.calls` is a `list[ToolCall]`; each `ToolCall` has `name: str`, `args: dict`, `ok: bool`, `seconds: float`, `result_chars: int`, `error: str`. `ws.log.names()` is the list of tool names in call order.

The evidence bundle —

`A.gather_evidence(submission_id, ws=None) -> str`

Runs `load_submission`, `submission_record`, `graph_metrics`, `spectrum_position`, `scan_smells` (five logged calls, in `ws` if you pass one) and returns **JSON text**:

```
{"submission": {"submission": str, "team": str,
"declared_level": str, "purpose": str},
"metrics": {"classes": int, "subclass_axioms": int, ...},
"spectrum": {"level": str, "evidence": [str, ...]},
"scanner_findings": [{"smell": str, "subject": str, "detail":
str}, ...]}
```

Course library calls used here

call	returns / notes
<code>ev.ScoreReport(score, notes, violated)</code>	<code>score: float</code> in <code>[0, 1]</code> ; <code>notes: list[str]</code> ; <code>violated: list[str]</code> of rulebook ids
<code>ev.parse_label_list(value)</code>	<code>list[str]</code> from a list, a JSON array string <code>'["a","b"]'</code> , a comma/space-separated string, or <code>None</code> → <code>[]</code>
<code>ev.set_f1(predicted, gold)</code>	(precision, recall, f1) floats over two iterables of labels; both empty → <code>(1.0, 1.0, 1.0)</code>
<code>ev.evaluate_dataset(program, dataset, scorer)</code>	<code>{"mean_score": float, "n": int, "violations": {rule_id: count}, "rows": [{"item": id, "score": float, "violated": [...]}]}</code> — one LM call per row
<code>ev.make_gepa_metric(scorer, rulebook)</code>	a GEPA metric: (gold, pred, trace=None, pred_name=None, pred_trace=None) -> <code>dspy.Prediction(score=..., feedback=...)</code>
<code>ev.judge(task, gold, report)</code>	<code>ScoreReport</code> from the configured DSPy LM: <code>.score</code> in <code>[0, 1]</code> , <code>.notes[0]</code> the critique. One billed call
<code>llm.configure_dspy()</code>	installs Claude as DSPy's default LM and returns it (<code>lm</code>)

call	returns / notes
<code>llm.dspy_lm(cache=False) / llm.reflection_lm()</code>	a separate <code>dspy.LM</code> (uncached / for GEPA reflection)
<code>with llm.meter(lm, ...) as cost:</code>	after the block, <code>cost == {"calls": int, "input_tokens": int, "output_tokens": int, "usd": float}</code>
<code>llm.chat_usage(messages)</code>	<code>{"model_turns", "input_tokens", "output_tokens", "usd_estimate"}</code> for a LangChain transcript
<code>opt.run_gepa(program, trainset, metric, *, valset, max_metric_calls, reflection_lm)</code>	the optimised program (a <code>dspy.Module</code>)
<code>opt.compare(before, after, dataset, scorer)</code>	<code>OptimisationResult</code> : <code>.before</code> / <code>.after</code> (evaluate_dataset dicts), <code>.instruction_before</code> , <code>.instruction_after</code> , <code>.delta</code> , <code>.report()</code>
<code>opt.instruction_of(program)</code>	the instruction text of a single-predictor program
<code>agents.build_agent(ws, system_prompt=..., tools=...)</code>	(agent, ws) — a LangChain tool-calling agent on Claude
<code>agents.run_agent(agent, ws, task)</code>	<code>AgentRun</code> : <code>.answer</code> (final text), <code>.messages</code> (transcript), <code>.log</code> (= <code>ws.log</code>)
<code>config.artifacts_dir()</code>	Path to <code>course/artifacts/</code> (created on first use)

The MDP (`mdp`) and skill (`Skill` , `SelfImprovingSkill`) APIs are described in the D2 and D3 contracts, where they are used.

Run the next cell to see a real row, real tool output and a real evidence bundle — no API calls are made.

```
In [ ]: ex = ROW["safety-incidents"]
print("row:", {k: ex[k] for k in ("id", "declared_level", "level", "raw_smells", "s
print("program input:", ex.inputs())

ws_ref = A.RegistryWorkspace()
tools_ref = {t.name: t for t in A.build_registry_tools(ws_ref)}
print("\nload_submission ->", tools_ref["load_submission"].invoke({"submission_id":
print("spectrum_position ->", json.loads(tools_ref["spectrum_position"].invoke({}))
print("scan_smells ->", [f["smell"] for f in json.loads(tools_ref["scan_smells"].in
print("log ->", [(c.name, c.ok, c.result_chars) for c in ws_ref.log.calls])

evidence_ref = json.loads(A.gather_evidence(ex.id))
print("\nevidence keys:", list(evidence_ref), "| declared:", evidence_ref["submissi
```

Part A — The task, its tools, and a deterministic control

Before a model is involved, pin down what the task *is*. Three clauses of the policy are decidable from tool output; one (the report) is not. Coding the decidable ones gives you a reference implementation, a way to validate the gold labels, and the control that any Claude system must be compared with.

A1. The intake policy as code (10 pts)

Implement, from the policy text (not from the gold labels):

- `reportable_smells(raw, measured_level) -> list[str]` — clause 2: the sorted scanner ids that must be reported, after waiver W1;
- `intake_decision(reported, declared_level, measured_level) -> str` — clause 3: "accept", "revise" or "reject". Use `ont.SPECTRUM_LEVELS` for the level order.

The check runs both on all 24 submissions against the board's labels, and on edge cases the corpus does not contain.

In writing: (a) the policy keys waiver W1 on the *measured* level, not the declared one — give the argument for that choice, and one submission in the corpus whose decision would change if it were keyed on the declared level; (b) name one question the policy text leaves open that you had to decide in code; (c) what does this exercise tell you about asking an LLM to "apply the policy"?

Background and design decisions — A1

You are turning prose into a **reference implementation**. It serves three purposes later: it validates the gold labels, it becomes the deterministic *control* (A2), and it is the engine of the critic (D1) and the MDP (D2). So it must follow the **policy text**, not the labels. If you tune it until the labels match, you can no longer detect a wrong label.

Decisions you make:

- **Which level the waiver keys on.** The policy says *measured*; the written part asks you to argue why that is the right design, not merely what the text says.
- **Rule order.** Clause 3 is ordered: *reject* beats *revise* beats *accept*. Encoding it as ordered checks makes the precedence explicit; a points-based encoding hides it.
- **Level comparison.** "Below the declared level" needs an order; derive it from `ont.SPECTRUM_LEVELS` rather than hard-coding one, so the policy and the classifier cannot drift apart.
- **Purity.** No tools and no model inside these functions: they are called hundreds of times in D2, and they must give the same answer every time.

Contract — A1

```
def reportable_smells(raw: Iterable[str], measured_level: str) ->
list[str]
def intake_decision(reported: Iterable[str], declared_level: str,
measured_level: str) -> str
```

`reportable_smells` — policy clause 2.

parameter	type	value in the corpus	meaning
<code>raw</code>	<code>Iterable[str]</code> (list or set)	<code>ex.raw_smells</code>	smell ids the scanner found
<code>measured_level</code>	<code>str</code> , one of <code>ont.SPECTRUM_LEVELS</code>	<code>ex.level</code>	the level the classifier measured

Returns a **sorted** `list[str]` with no duplicates: every id in `raw` , except `"no-disjointness"` when `measured_level` is in `A.WAIVABLE_AT` . Never add an id that is not in `raw` ; do not modify `raw` . The check compares with `==` against `ex.smells` , so a set or an unsorted list fails.

`intake_decision` — policy clause 3.

parameter	type	value in the corpus	meaning
<code>reported</code>	<code>Iterable[str]</code>	output of <code>reportable_smells</code>	the smells being reported (already waived)
<code>declared_level</code>	<code>str</code>	<code>ex.declared_level</code>	what the team claimed
<code>measured_level</code>	<code>str</code>	<code>ex.level</code>	what the classifier measured

Returns exactly one of `"accept"` , `"revise"` , `"reject"` , decided **in this order**: `"reject"` if `reported` contains any id in `A.BLOCKING_SMELLS` ; else `"revise"` if `reported` is non-empty **or** `measured_level` comes *before* `declared_level` in `ont.SPECTRUM_LEVELS` ; else `"accept"` .

call	returns
<code>reportable_smells({"no-disjointness", "missing-label"}, "controlled-vocabulary")</code>	<code>["missing-label"]</code>
<code>reportable_smells(["no-disjointness"], "formal-ontology")</code>	<code>["no-disjointness"]</code>
<code>intake_decision(["subsumption-cycle", "missing-label"], "formal-ontology", "taxonomy")</code>	<code>"reject"</code>

call	returns
<code>intake_decision([], "thesaurus", "taxonomy")</code>	"revise" (over-promises)
<code>intake_decision([], "controlled-vocabulary", "formal-ontology")</code>	"accept" (exceeds the declaration)

Both functions are pure: no tools, no model. Later problems (A2, D1, D2) reuse them.

```
In [ ]: def reportable_smells(raw: Iterable[str], measured_level: str) -> list[str]:
        """Policy clause 2 (see Contract A1): sorted reportable smell ids after waiver
        # TODO
        raise NotImplementedError

def intake_decision(reported: Iterable[str], declared_level: str, measured_level: str):
    """Policy clause 3 (see Contract A1): "accept", "revise" or "reject"."""
    # TODO
    raise NotImplementedError
```

```
In [ ]: # Marking scheme for A1 -- do not edit.
with GRADER.check('A1', points=6):
    for ex in ALL:
        mine = reportable_smells(ex.raw_smells, ex.level)
        assert mine == ex.smells, (ex.id, mine, ex.smells)
        assert intake_decision(mine, ex.declared_level, ex.level) == ex.decision, ex
    assert reportable_smells({"no-disjointness", "missing-label"}, "controlled-vocabulary") == ["no-disjointness", "missing-label"]
    assert reportable_smells({"no-disjointness"}, "formal-ontology") == ["no-disjointness"]
    assert intake_decision([], "controlled-vocabulary", "formal-ontology") == "accept"
    assert intake_decision(["undeclared-term"], "taxonomy", "formal-ontology") == "reject"
    assert intake_decision(["missing-label"], "taxonomy", "taxonomy") == "revise"
    assert intake_decision(["subsumption-cycle", "missing-label"], "formal-ontology", "formal-ontology") == "reject"
    assert intake_decision([], "thesaurus", "taxonomy") == "revise"
```

Your answer:

(Write your answer here.)

A2. Tools, trajectories, and what a mega-tool hides (8 pts)

The registry tools (`A.build_registry_tools(ws)`) are narrow and logged: every call lands in `ws.log` (a `ToolCallLog`). `A.TOOL_TO_EVIDENCE` names the kind of evidence each tool buys.

1. Write `evidence_from_log(log) -> frozenset` — the evidence kinds a trajectory bought.
2. Write `scripted_review(submission_id, ws) -> dict` with keys `level`, `smells`, `decision`: a *deterministic reviewer* that invokes (with `.invoke`) only the registry tools it needs and applies your A1 functions. Run it on all 24 submissions, each in a fresh workspace, and set `control_accuracy` to the fraction it gets fully right.

3. Build `a2`, one row per toolset for the submission `"interlocking-import"`: the decomposed trajectory of your scripted reviewer and one call of the monolithic `A.build_monolithic_tool(ws)`; columns `toolset` (`"decomposed"` / `"monolithic"`), `calls`, `result_chars` (sum over the log), `evidence_visible` (`sorted(evidence_from_log(...))`).

In writing: what does the monolithic tool cost you — think of the trajectory as a record, of reward shaping, and of least privilege — and when would you still ship one? Read the tool descriptions in `ch01_agentic.py`: which sentence in `submission_record`'s description makes an agent call it, and what would happen if it were missing?

Background and design decisions — A2

This is where you meet the **workspace** and the **tool registry** (Background §2–4) as a user. Build the tools on a workspace, call them with `.invoke`, decode the JSON, and look at `ws.log`. The scripted reviewer shows what the task costs *without any model*: if code gets 24/24 with four calls, every Claude result later is measured against that.

Decisions you make:

- **Which tools to call.** Only what the policy needs: the record (declared level), the spectrum (measured level), the scanner. `graph_metrics` is not needed to decide; every extra call is cost and noise in the trajectory.
- **A fresh workspace per submission**, so each log is one clean trajectory and nothing from the previous item leaks into the next.
- **What counts as evidence.** `evidence_from_log` counts only successful calls: a failed call bought nothing. `load_submission` buys no evidence; it only makes the others possible.
- **Narrow vs monolithic.** The monolithic tool is *one* call that returns everything, so the log cannot say what the decision used. The written part asks when you would still ship one.

Contract — A2

```
def evidence_from_log(log: ToolCallLog) -> frozenset[str]
def scripted_review(submission_id: str, ws: A.RegistryWorkspace) -> dict
control_accuracy: float
a2: pd.DataFrame

evidence_from_log(log) — log is a workspace's ws.log. Loop over log.calls; for
every call with ok == True whose name is a key of A.TOOL_TO_EVIDENCE, collect
A.TOOL_TO_EVIDENCE[name]. Return the kinds as a frozenset[str] (a subset of
A.EVIDENCE_KINDS). Example: a log of load_submission, spectrum_position,
scan_smells → frozenset({"spectrum", "smells"}).
```

`scripted_review(submission_id, ws)` — a deterministic reviewer, no model.

parameter	type	meaning
submission_id	str	e.g. "signalling-equipment"
ws	A.RegistryWorkspace	a fresh workspace; build the tools on it so every call lands in <code>ws.log</code>

It must call `load_submission` first, then only the tools whose output it needs (`submission_record` for the declared level, `spectrum_position` , `scan_smells`), **at most 5 calls in total, none failing**. Returns a dict with **exactly** these three keys (the check compares the whole dict with `==`):

```
{"level": "taxonomy",                                # str, the
measured level
  "smells": ["no-disjointness", "subsumption-cycle"], #
reportable_smells(...) output
  "decision": "reject"}                               #
intake_decision(...) output
```

Remember that `scan_smells` returns one entry per finding, so collect the `"smell"` values into a set before calling `reportable_smells` .

control_accuracy — float in [0, 1]: over all 24 rows of `ALL` , the fraction for which `scripted_review(ex.id, A.RegistryWorkspace()) == {"level": ex.level, "smells": ex.smells, "decision": ex.decision}` .

a2 — a 2-row DataFrame for submission `"interlocking-import"` :

column	type	meaning
toolset	str	"decomposed" (your <code>scripted_review</code>) or "monolithic" (one <code>review_submission</code> call)
calls	int	<code>len(ws.log.calls)</code> for that toolset's workspace
result_chars	int	<code>sum(c.result_chars for c in ws.log.calls)</code>
evidence_visible	list[str]	<code>sorted(evidence_from_log(ws.log))</code>

Use a separate fresh workspace for each row. The monolithic tool is

```
A.build_monolithic_tool(ws)[0].invoke({"submission_id": "interlocking-import"}) .
```

```
In [ ]: ws_demo = A.RegistryWorkspace()
for t in A.build_registry_tools(ws_demo):
    params = list(t.args_schema.model_json_schema().get("properties", {}))
    print(f"{t.name:18s} {str(params):18s} {t.description.splitlines()[0][:80]}")
```

```
In [ ]: def evidence_from_log(log) -> frozenset[str]:
        """Evidence kinds bought by the ok calls in a ToolCallLog (see Contract A2)."""
        # TODO
        raise NotImplementedError
```

```
def scripted_review(submission_id: str, ws) -> dict:
    tools = {t.name: t for t in A.build_registry_tools(ws)}
    """Deterministic review -> {"level", "smells", "decision"} (see Contract A2)."""
    # TODO: invoke only the tools you need, then apply reportable_smells / intake_d
    raise NotImplementedError

control_accuracy = None # TODO
a2 = None # TODO
```

```
In [ ]: # Marking scheme for A2 -- do not edit.
with GRADER.check('A2', points=4):
    ws_c = A.RegistryWorkspace()
    out = scripted_review("signalling-equipment", ws_c)
    assert out == {"level": "taxonomy", "smells": ["no-disjointness", "subsumption-
        decision": "reject"], out
    assert not any(not c.ok for c in ws_c.log.calls), "no failed tool calls"
    assert len(ws_c.log.calls) <= 5, "call only the tools you need"
    assert evidence_from_log(ws_c.log) >= {"spectrum", "smells"}
    assert control_accuracy == 1.0
    assert isinstance(a2, pd.DataFrame) and set(a2["toolset"]) == {"decomposed", "m
    m = a2.set_index("toolset")
    assert m.loc["monolithic", "calls"] == 1 and list(m.loc["monolithic", "evidence
    assert {"spectrum", "smells"} <= set(m.loc["decomposed", "evidence_visible"])
```

Your answer:

(Write your answer here.)

Part B — Build the grader, the Claude program, and validate the judge

The guidelines a scorer can report are in `A.INTAKE_RULEBOOK` — ids plus the sentence GEPA's reflection step will read:

```
In [ ]: for rule in A.INTAKE_RULEBOOK:
    print(f"{rule.id:24s} {rule.description[:95]}")
```

B1. A diagnostic triage scorer (12 pts)

Implement `intake_scorer(gold, pred) -> ev.ScoreReport`. `gold` is a dataset row (`level`, `raw_smells`, `smells`, `decision`); `pred` has `level`, `smells` (anything `ev.parse_label_list` accepts), `decision`, `report`.

`score = 0.3 * level_ok + 0.4 * smell_f1 + 0.3 * decision_ok`, where `level` and `decision` are compared after `strip().lower()`, and `smell_f1` is `ev.set_f1(pred, gold.smells)`. Violations (list them in `A.INTAKE_RULEBOOK` order, without duplicates):

condition	violated
level or decision not in the allowed labels	use-label-vocabulary
a valid level, but wrong	cite-spectrum-evidence
a gold smell not reported	report-all-smells
a reported smell the scanner found but the policy waives	apply-waivers
a reported smell the scanner did not find	no-unsupported-claims
a valid decision, but wrong	decide-by-policy
an empty report (no score change)	ground-in-tool-output

`notes` must say what was expected for every wrong field — that text is what GEPA reads. Why separate `apply-waivers` from `no-unsupported-claims`? Both are "reported something not in gold", but they call for different fixes, and feedback that cannot tell them apart cannot teach either.

Background and design decisions — B1

The scorer is the most reused piece of the problem set: it grades the baseline, it is GEPA's objective, it scores the agent, and it is the MDP's reward. Its **notes** are also what GEPA's reflection reads (Background §7–8), so write them for a reader who will rewrite the prompt.

Decisions you make:

- **Weights.** 0.3 / 0.4 / 0.3 is given; notice what it implies: a perfect decision with the wrong level still earns 0.7.
- **Partial credit for smells** via F1 rather than all-or-nothing: an optimiser gets a gradient (reporting 2 of 3 defects beats reporting none), and precision and recall errors stay separate.
- **Normalisation.** Accept `" Taxonomy "` but not `"hierarchy"`: be tolerant about format and strict about vocabulary.
- **Distinct violation ids for distinct fixes:** `apply-waivers` (knows the scanner, not the policy) versus `no-unsupported-claims` (invents defects). If the feedback merges them, the optimiser cannot learn either fix.
- **The report does not change the score** but still records a violation: the report is graded elsewhere (B3); here you only notice that it is missing.

Contract — B1

```
WEIGHTS = {"level": 0.3, "smells": 0.4, "decision": 0.3}
def intake_scorer(gold: dspy.Example, pred) -> ev.ScoreReport
```

parameter	type	fields read	meaning
gold	dspy.Example (a dataset row)	level , raw_smells , smells , decision	the board's answer
pred	any object with attributes (usually dspy.Prediction)	level , smells , decision , report	the system's answer

Read pred defensively: a field may be missing, `None`, or oddly formatted. Use `getattr(pred, "level", "")`; normalise `level` and `decision` with `str(...).strip().lower()`; parse `smells` with `ev.parse_label_list(...)` (it accepts a list, a JSON string such as `'["a", "b"]'`, a comma-separated string, or `None`); treat `report` as empty when `str(...).strip() == ""`.

Returns `ev.ScoreReport(score, notes, violated)`:

- `score: float = 0.3 * level_ok + 0.4 * smell_f1 + 0.3 * decision_ok`, where `level_ok` and `decision_ok` are 1.0 or 0.0 and `smell_f1 = ev.set_f1(pred_smells, gold.smells)[2]`. The report never changes the score.
- `violated: list[str]` — the rule ids from the table in the brief, **each at most once, in A.INTAKE_RULEBOOK.ids order**. Only the ids in that rulebook are allowed.
- `notes: list[str]` — one sentence per wrong field that states the **expected** value (for example the correct decision); a final summary line is fine.

How to split "reported something not in gold": let `extra = pred_smells - set(gold.smells)`. Ids in `extra` `n` `gold.raw_smells` were found but waived → `apply-waivers`; ids in `extra - gold.raw_smells` were never found → `no-unsupported-claims`.

gold row	prediction	score	violated
signalling-equipment	perfect	1.0	<code>[]</code>
same	<code>level="hierarchy"</code>	0.7	<code>["use-label-vocabulary"]</code>
same	<code>smells=["subsumption-cycle"]</code> (misses one of two)	$0.3 + 0.4 \cdot (2/3) + 0.3$	<code>["report-all-smells"]</code>
same	perfect, but <code>report=""</code>	1.0	<code>["ground-in-tool-output"]</code>

```
In [ ]: WEIGHTS = {"level": 0.3, "smells": 0.4, "decision": 0.3}

def intake_scorer(gold: dspy.Example, pred) -> ev.ScoreReport:
    """0.3*level + 0.4*smell F1 + 0.3*decision, with rulebook violations (see Contr
    # TODO
    raise NotImplementedError
```

```
In [ ]: # Marking scheme for B1 -- do not edit.
with GRADER.check('B1', points=10):
    P = lambda **kw: dspy.Prediction(**{"level": "taxonomy", "decision": "reject",
                                         "smells": ["subsumption-cycle", "no-disjoin
sig, thes = ROW["signalling-equipment"], ROW["timetable-service-thesaurus"]
cases = [
    (sig, P(), 1.0, []),
    (sig, P(level="Taxonomy", smells=["no-disjointness", "subsumption-cycle"],
    (sig, P(level="hierarchy"), 0.7, ["use-label-vocabulary"]),
    (sig, P(level="thesaurus"), 0.7, ["cite-spectrum-evidence"]),
    (sig, P(smells=["subsumption-cycle"], 0.3 + 0.4 * 2 / 3 + 0.3, ["report-al
    (sig, P(smells=["subsumption-cycle", "no-disjointness", "missing-label"]),
    ["no-unsupported-claims"]),
    (sig, P(decision="revise"), 0.7, ["decide-by-policy"]),
    (sig, P(decision="REJECTED"), 0.7, ["use-label-vocabulary"]),
    (sig, P(report=""), 1.0, ["ground-in-tool-output"]),
    (thes, P(level="thesaurus", smells=["no-disjointness"], decision="accept"),
    ["apply-waivers"]),
    (thes, P(level="taxonomy", smells=["no-disjointness", "missing-label"], dec
    0.0, ["cite-spectrum-evidence", "apply-waivers", "no-unsupported-claims",
    "decide-by-policy", "ground-in-tool-output"]),
]
for gold, pred, score, violated in cases:
    r = intake_scorer(gold, pred)
    assert abs(r.score - score) < 1e-9 and r.violated == violated, (gold.id, di
    assert all(v in A.INTAKE_RULEBOOK for v in r.violated)
r = intake_scorer(sig, P(decision="revise"))
assert any("reject" in n for n in r.notes), "say what was expected"
```

B2. The Claude triage program, and its baseline (10 pts)

Tools measure, the model interprets: the program gathers evidence deterministically
(`A.gather_evidence(submission)` — record, metrics, spectrum, scanner findings) and
makes **one** LM call.

1. Write a signature `IntakeTriage` with input `evidence` and outputs `level`,
`smells` (`list[str]`), `decision`, `report`, and a factory
`TriageProgram(instruction=BASELINE_INSTRUCTION)` returning a `dspy.Module`
with a single `dspy.Predict` whose forward takes `submission`. Field descriptions are
part of the prompt — write them as you would brief a colleague (manual marks). The
baseline instruction deliberately does **not** contain the policy.
2. Configure Claude (`lm = llm.configure_dspy()`), run once on `train[0]` (`smoke`).
3. Evaluate on `dev` with `ev.evaluate_dataset` inside `llm.meter(lm)`; store
`baseline_dev` and `baseline_dev_cost`.

In writing: an error analysis of the baseline — for each non-perfect item, which guideline and
why you think Claude did it — and compare with the control (A2). Which errors come from
not knowing the policy, and which from *not reading the evidence*?

Background and design decisions — B2

This is the "tools measure, model interprets" design (Background §5–6): deterministic evidence gathering, then **one** Claude call through a DSPy predictor. The baseline instruction deliberately omits the policy, so the baseline shows how Claude behaves *without* the rules, and Part C shows how much writing them down is worth.

Decisions you make:

- **Signature fields and descriptions.** They are part of the prompt. Listing the allowed labels in a field description ("one of: accept, revise, reject") is cheap and often fixes vocabulary errors before any optimisation.
- **Output types.** `smells: list[str]` makes DSPy ask for, and parse, a list; a `str` field would leave the parsing to you (and to `ev.parse_label_list`).
- **One predictor, not several.** One instruction is one thing to optimise and one thing to attribute a change to. Splitting level, smells and decision into three calls would triple cost and make GEPA's job harder.
- **What goes into the prompt.** The whole evidence bundle, not the Turtle source: the model interprets measurements; it does not re-derive them from raw RDF.

Contract — B2

```
BASELINE_INSTRUCTION: str # given; do not add the
policy to it
class IntakeTriage(dspy.Signature): ...
def TriageProgram(instruction: str = BASELINE_INSTRUCTION) -> dspy.Module
lm, smoke, baseline_dev, baseline_dev_cost
IntakeTriage — exactly one input field and four output fields:
```

field	kind	type	content
evidence	dspy.InputField	str	the JSON text from <code>A.gather_evidence(...)</code>
level	dspy.OutputField	str	one spectrum level
smells	dspy.OutputField	list[str]	smell ids to report
decision	dspy.OutputField	str	accept / revise / reject
report	dspy.OutputField	str	a short report to the team

TriageProgram(instruction) returns a `dspy.Module` that:

- holds **exactly one** `dspy.Predict` on `IntakeTriage` whose instruction is `instruction` (use `IntakeTriage.with_instructions(instruction)`), so that `opt.instruction_of(TriageProgram("x")) == "x"`;
- has `forward(self, submission: str) -> dspy.Prediction`, which calls `A.gather_evidence(submission)` and passes the result as `evidence=`.

It is called as `program(**ex.inputs())`, i.e. `program(submission="station-facility-types")`, and returns a prediction with `.level`, `.smells`, `.decision`, `.report`.

Variables the check reads

name	type	how to produce it
<code>lm</code>	<code>dspy.LM</code>	<code>llm.configure_dspy()</code>
<code>smoke</code>	<code>dspy.Prediction</code>	<code>TriageProgram()(**train[0].inputs())</code> — one billed call
<code>baseline_dev</code>	<code>dict</code>	<code>ev.evaluate_dataset(TriageProgram(), dev, intake_scorer)</code> (shape in the Reference)
<code>baseline_dev_cost</code>	<code>dict</code>	with <code>llm.meter(lm)</code> as <code>baseline_dev_cost:</code> around the evaluation

```
In [ ]: BASELINE_INSTRUCTION = ("You review ontology submissions to Meridian Rail's ontolog
                                "registry. From the evidence, give the submission's level,
                                "defects to report, the intake decision and a report to the

# TODO: class IntakeTriage(dspy.Signature): ...
# TODO: def TriageProgram(instruction=BASELINE_INSTRUCTION): ...

lm = llm.configure_dspy()
smoke = None # TODO
# TODO: baseline_dev, baseline_dev_cost

raise NotImplementedError
```

```
In [ ]: # Marking scheme for B2 -- do not edit.
with GRADER.check('B2', points=4):
    assert set(IntakeTriage.input_fields) == {"evidence"}
    assert {"level", "smells", "decision", "report"} <= set(IntakeTriage.output_fields)
    program = TriageProgram("custom instruction")
    assert len(list(program.named_predictors())) == 1
    assert opt.instruction_of(program) == "custom instruction"
    assert isinstance(smoke.level, str) and smoke.level.strip()
    assert baseline_dev["n"] == len(dev) == 8
    assert {r["item"] for r in baseline_dev["rows"]} == {ex.id for ex in dev}
    assert {"calls", "usd"} <= set(baseline_dev_cost)
```

Your answer:

(Write your answer here.)

B3. Validate the LLM judge before trusting it (10 pts)

The office wants to grade the free-text reports with an LLM judge (`ev.judge`). A judge is itself a model: it has to be validated against human labels before its scores mean anything.

A. LABELLED_REPORTS holds 12 reports the board's reviewers labelled *grounded* (every claim supported by the tool output, with specific terms or counts) or not — half of each.

1. Implement `judge_agreement(labels, scores, threshold=0.5) -> dict` with keys `n`, `accuracy`, `precision`, `recall` (positive class = grounded; predicted grounded iff `score >= threshold`; precision/recall are 0.0 when undefined) and `kappa` (Cohen's κ ; when chance agreement is 1, return 1.0 if observed agreement is 1, else 0.0).
2. Run `ev.judge(task, gold, report)` on every labelled report inside `llm.meter(1m)` — `task` describes the intake review, `gold` is the JSON of that submission's gold `level` / `smells` / `decision`. Build `b3` with columns `report`, `submission`, `label`, `judge_score`, and set `b3_stats = judge_agreement(...)` at 0.5 and `b3_cost`.
3. Build `b3_thresholds`: one row per threshold in `[0.3, 0.5, 0.7]` with the agreement statistics.

In writing: is this judge fit to grade the reports, on this evidence? What does `n = 12` let you conclude? Which kinds of ungrounded report would you expect it to miss, and what would you change before using it?

Background and design decisions — B3

The judge (Background §9) will grade the free-text reports, the one field no function can score. Before trusting it, treat it as a classifier and measure it against the board's labels. The labelled set is small and balanced (6 grounded, 6 not) on purpose.

Decisions you make:

- **Metric.** Report κ as well as accuracy: on a balanced set a judge that always says "grounded" gets 0.5 accuracy but $\kappa = 0$.
- **Threshold.** The judge returns a number; "grounded" is a threshold choice. The threshold table shows the trade between catching fabrications and failing good reports.
- **What the judge sees.** Here it gets the gold labels, not the tool output, so it cannot check a count. The written part asks what you would change before relying on it.
- **What `n = 12` supports.** Very little; say so.

Contract — B3

```
def judge_agreement(labels: Iterable[bool], scores: Iterable[float],
threshold: float = 0.5) -> dict
b3: pd.DataFrame; b3_stats: dict; b3_cost: dict; b3_thresholds:
pd.DataFrame
```

`judge_agreement` — pure arithmetic, no model.

parameter	type	meaning
labels	Iterable[bool] (list or pd.Series)	human verdict per report: True = grounded
scores	Iterable[float] in [0, 1], same length	the judge's score per report
threshold	float	predicted grounded iff score >= threshold

Returns {"n": int, "accuracy": float, "precision": float, "recall": float, "kappa": float} , with grounded as the positive class. $\text{precision} = \text{TP} / (\text{TP} + \text{FP})$ and $\text{recall} = \text{TP} / (\text{TP} + \text{FN})$, each 0.0 when its denominator is 0. Cohen's $\text{kappa} = (\text{p}_o - \text{p}_e) / (1 - \text{p}_e)$, where p_o is the observed agreement (= accuracy) and $\text{p}_e = P(\text{pred grounded}) \cdot P(\text{label grounded}) + P(\text{pred not}) \cdot P(\text{label not})$; when $\text{p}_e == 1$, return 1.0 if $\text{p}_o == 1$ else 0.0 . Example: `judge_agreement([True, True, False, False], [0.9, 0.4, 0.2, 0.6])` → {"n": 4, "accuracy": 0.5, "precision": 0.5, "recall": 0.5, "kappa": 0.0} .

Data. A.LABELLED_REPORTS is a list of 12 tuples (report_id: str, submission_id: str, text: str, grounded: bool) . The judge call is `ev.judge(task, gold, text)` , where `task` is a str describing the intake review and `gold` is `json.dumps({"level": ..., "smells": [...], "decision": ...})` for that submission (take the values from `ROW[submission_id]`). It returns a `ScoreReport` : use `.score` .

name	type	content
b3	DataFrame, 12 rows	columns report (id), submission , label (bool), judge_score (float in [0, 1])
b3_stats	dict	exactly <code>judge_agreement(b3.label, b3.judge_score)</code> (threshold 0.5)
b3_cost	dict	<code>llm.meter(1m)</code> around the 12 judge calls
b3_thresholds	DataFrame, 3 rows	column threshold = [0.3, 0.5, 0.7] plus the five judge_agreement keys

```
In [ ]: def judge_agreement(labels: Iterable[bool], scores: Iterable[float], threshold: float) -> dict:
    """{"n", "accuracy", "precision", "recall", "kappa"} (see Contract B3)."""
    # TODO
    raise NotImplementedError

b3, b3_stats, b3_cost, b3_thresholds = None, None, None, None # TODO
```

```
In [ ]: # Marking scheme for B3 -- do not edit.
with GRADER.check('B3', points=5):
    s = judge_agreement([True, True, False, False], [0.9, 0.4, 0.2, 0.6])
    assert s["n"] == 4 and abs(s["accuracy"] - 0.5) < 1e-9 and abs(s["kappa"]) < 1e-9
```

```

assert abs(s["precision"] - 0.5) < 1e-9 and abs(s["recall"] - 0.5) < 1e-9
s = judge_agreement([True, True, False, False, False], [0.9, 0.6, 0.4, 0.2, 0.7])
assert abs(s["accuracy"] - 0.8) < 1e-9 and abs(s["kappa"] - 0.32 / 0.52) < 1e-9
assert abs(s["precision"] - 2 / 3) < 1e-9 and s["recall"] == 1.0
assert judge_agreement([True, True], [0.9, 0.8])["kappa"] == 1.0
assert judge_agreement([False, False], [0.9, 0.1], 0.5)["precision"] == 0.0
assert isinstance(b3, pd.DataFrame) and len(b3) == len(A.LABELLED_REPORTS) == 1
assert {"report", "submission", "label", "judge_score"} <= set(b3.columns)
assert all(0.0 <= x <= 1.0 for x in b3.judge_score)
assert b3_stats == judge_agreement(b3.label, b3.judge_score, 0.5)
assert list(b3_thresholds["threshold"]) == [0.3, 0.5, 0.7] and {"accuracy", "kappa"} <= set(b3_stats)
assert {"calls", "usd"} <= set(b3_cost)

```

Your answer:

(Write your answer here.)

Part C — Optimise with GEPA — and report it honestly

GEPA rewrites the instruction by reflecting on the scorer's feedback. It is only as good as that feedback, it overfits small training sets, and every metric call is billed. The rules for this part:

optimise on train, select on dev, report on test — with a cost and a noise estimate next to every number.

C1. A budgeted GEPA run with a held-out report (10 pts)

1. Build the feedback metric from your scorer and `A.INTAKE_RULEBOOK` (`ev.make_gepa_metric`) and a reflection LM (`llm.reflection_lm()`).
2. Run `opt.run_gepa` on `train` with `valset=dev` and `max_metric_calls=GEPA_BUDGET` inside `llm.meter(lm, reflect)`; store the cost in `gepa_cost` and the program in `tuned`.
3. Compare baseline and tuned on `test` with `opt.compare` (`c1`); print `c1.report()`.
4. Save the tuned instruction to `config.artifacts_dir() / "ch01_registry_triage_instruction.txt"`.

In writing: read the instruction diff. Which policy clauses did GEPA write down, which did it miss, and why? (Look at which blocking defects occur in `train`.) Did it add anything that is not policy?

Background and design decisions — C1

GEPA (Background §8) rewrites the B2 instruction using your B1 feedback. The three splits have three jobs: **train** is what GEPA learns from, **dev** is what it uses to choose between

candidate instructions, and **test** is only for the report. Using test for anything else makes the reported number optimistic.

Decisions you make:

- **Budget.** `max_metric_calls` is the cost dial: each call is one billed program run. 60 on 8 training items is enough for a few reflection rounds; more mostly buys over-fitting.
- **Which LMs to meter.** GEPA uses the task LM *and* the reflection LM; meter both, or the cost line is wrong.
- **The instruction is an artefact.** Save it, read the diff, and treat it like code under review: it will run on every future call.

Contract — C1

name	type	how to produce it
<code>GEPA_BUDGET</code>	<code>int</code> ≤ 100	given (60): the maximum number of metric calls GEPA may make
<code>gepa_metric</code>	callable	<code>ev.make_gepa_metric(intake_scorer, A.INTAKE_RULEBOOK)</code>
<code>reflect</code>	<code>dspy.LM</code>	<code>llm.reflection_lm()</code> — the model that proposes new instructions
<code>tuned</code>	<code>dspy.Module</code>	<code>opt.run_gepa(TriageProgram(), train, gepa_metric, valset=dev, max_metric_calls=GEPA_BUDGET, reflection_lm=reflect)</code>
<code>gepa_cost</code>	<code>dict</code>	<code>with llm.meter(lm, reflect) as gepa_cost:</code> around <code>run_gepa</code> (meter both LMs)
<code>c1</code>	<code>OptimisationResult</code>	<code>opt.compare(TriageProgram(), tuned, test, intake_scorer)</code> — on test , never on <code>train</code> / <code>dev</code>
<code>instruction_path</code>	<code>pathlib.Path</code>	<code>config.artifacts_dir() / "ch01_registry_triage_instruction.txt"</code> , whose content must equal <code>opt.instruction_of(tuned)</code> <code>(instruction_path.write_text(..., encoding="utf-8"))</code>

`train` is what GEPA learns from; `dev` (`valset`) is what it uses to choose between candidate instructions; `test` is only for the report. `c1.report()` prints the before and after means, the violation histograms and the instruction diff.

```
In [ ]: GEPA_BUDGET = 60
# TODO: gepa_metric, reflect, tuned (inside llm.meter -> gepa_cost), c1, save the i
```

```
instruction_path = config.artifacts_dir() / "ch01_registry_triage_instruction.txt"

raise NotImplementedError
```

```
In [ ]: # Marking scheme for C1 -- do not edit.
with GRADER.check('C1', points=5):
    ids = lambda rows: {ex.id for ex in rows}
    assert not (ids(test) & (ids(train) | ids(dev))), "the test split must be untou
    assert {r["item"] for r in c1.after["rows"]} == ids(test), "report on the test
    assert c1.before["n"] == c1.after["n"] == len(test)
    assert instruction_path.is_file()
    assert instruction_path.read_text(encoding="utf-8") == opt.instruction_of(tuned
    assert {"calls", "usd"} <= set(gepa_cost) and GEPA_BUDGET <= 100
```

Your answer:

(Write your answer here.)

C2. Was the optimiser worth it — and was the feedback? (8 pts)

Four contenders on `test`, each evaluated inside its own `llm.meter(1m)`:

- `"baseline"`, `"gepa"` (from C1);
- `"guidelines"`:
`TriageProgram(A.INTAKE_RULEBOOK.as_guidelines(BASELINE_INSTRUCTION))`
 — the policy written into the prompt by hand;
- `"gepa-blind"`: GEPA with the same budget but a metric whose feedback is only the score (`f"Score: {s:.3f}"`) — the ablation that shows what the diagnostic feedback is worth. Meter its optimisation too.

Build `c2` with columns `program`, `test_mean`, `violations`, `eval_usd`, `optimisation_usd` (0 for the two unoptimised programs), `instruction_chars`.

In writing: which would you deploy, and what would change your mind? What does the blind run tell you about metric design?

Background and design decisions — C2

An optimiser's gain is only a finding if it beats the **cheap alternatives**. Two controls: the policy written into the prompt by hand (*guidelines*, free), and GEPA with a feedback-free metric (*blind*), which isolates what the diagnostic notes are worth.

Decisions you make:

- **What to deploy.** The cheapest program whose score is within the C3 noise of the best, counting both optimisation cost (once) and instruction length (paid on every call).

- **What the blind run tells you about metric design:** if it matches the diagnostic run, the notes were not doing the work; if it lags, they were.

Contract — C2

The **blind metric** has the same signature as a GEPA metric:

```
def blind_metric(gold, pred, trace=None, pred_name=None, pred_trace=None)
-> dspy.Prediction
    # returns dspy.Prediction(score=<intake_scorer score>,
feedback=f"Score: {score:.3f}")
```

Optimise it exactly like C1 (`train` , `valset=dev` , the same `GEPA_BUDGET` and `reflect`), metered with `llm.meter(lm, reflect)` .

c2 — a 4-row DataFrame, all evaluated on `test` , each inside its own `llm.meter(lm)` :

column	type	content
<code>program</code>	<code>str</code>	<code>"baseline"</code> , <code>"guidelines"</code> , <code>"gepa"</code> , <code>"gepa-blind"</code>
<code>test_mean</code>	<code>float</code>	<code>evaluate_dataset(...)["mean_score"]</code>
<code>violations</code>	<code>dict</code>	<code>evaluate_dataset(...)["violations"]</code>
<code>eval_usd</code>	<code>float</code>	the <code>"usd"</code> of that evaluation's meter
<code>optimisation_usd</code>	<code>float</code>	<code>0.0</code> for baseline and guidelines; <code>gepa_cost["usd"]</code> for gepa; the blind run's optimisation cost for gepa-blind
<code>instruction_chars</code>	<code>int</code>	<code>len(opt.instruction_of(program))</code>

`"guidelines"` is

`TriageProgram(A.INTAKE_RULEBOOK.as_guidelines(BASELINE_INSTRUCTION))` .

```
In [ ]: guidelines_instruction = A.INTAKE_RULEBOOK.as_guidelines(BASELINE_INSTRUCTION)
c2 = None # TODO

raise NotImplementedError
```

```
In [ ]: # Marking scheme for C2 -- do not edit.
with GRADER.check('C2', points=3):
    assert isinstance(c2, pd.DataFrame)
    assert set(c2["program"]) == {"baseline", "guidelines", "gepa", "gepa-blind"}
    assert {"test_mean", "violations", "eval_usd", "optimisation_usd", "instruction
    assert c2.set_index("program").loc[["baseline", "guidelines"], "optimisation_us
    assert c2.set_index("program").loc["gepa", "optimisation_usd"] == gepa_cost["us
```

Your answer:

(Write your answer here.)

C3. Is the difference bigger than the noise? (8 pts)

Eight test items at temperature 1.0: one item is 0.125 of the mean. Estimate the noise. With caching **disabled** (`fresh = llm.dspy_lm(cache=False)`), with `dspy.context(lm=fresh)`, run the baseline and the GEPA-tuned programs on `test` three times each; store the mean scores in `runs = {"baseline": [...], "gepa": [...]}` and the cost in `c3_cost`.

Set `verdict_c3` to `"significant"` if `|mean_gepa - mean_baseline| > 2 * max(sd_gepa, sd_baseline)` (sample sd), else `"not significant"`. In writing: what does this say about C1 and C2, and what would a credible evaluation of the triage system need?

Background and design decisions — C3

Claude samples at temperature 1.0, and each test split has 8 items, so a single number moves by 0.125 per item. Before claiming that GEPA helped, measure how much a score moves when **nothing** changes. DSPy caches identical requests, so repeated runs return the same answer unless you use an uncached LM (`cache=False`).

Decisions you make:

- **How many repeats.** Three is the minimum for a standard deviation; say what it can and cannot resolve.
- **The decision rule.** The rule here (difference greater than twice the larger sd) is deliberately simple; the written part asks what a credible evaluation would need instead.

Contract — C3

name	type	content
<code>fresh</code>	<code>dspy.LM</code>	<code>llm.dspy_lm(cache=False)</code> — an uncached LM, so repeated runs really re-sample
<code>runs</code>	<code>dict[str, list[float]]</code>	exactly the keys <code>"baseline"</code> and <code>"gepa"</code> , each a list of 3 mean test scores
<code>c3_cost</code>	<code>dict</code>	<code>llm.meter(fresh)</code> around all six evaluations
<code>verdict_c3</code>	<code>str</code>	<code>"significant"</code> or <code>"not significant"</code>

Run the evaluations inside `with dspy.context(lm=fresh):` so the programs use the uncached LM. Compute the mean and the **sample** standard deviation (`statistics.stdev`) of each list; the verdict is `"significant"` iff `abs(mean_gepa - mean_baseline) > 2 * max(sd_gepa, sd_baseline)`. The check recomputes it from `runs`, so the verdict must follow from the numbers you stored.

```
In [ ]: fresh = llm.dspy_lm(cache=False)
runs = {"baseline": [], "gepa": []}
# TODO: fill runs (inside llm.meter(fresh) as c3_cost), then compute verdict_c3
```

```

verdict_c3 = None

raise NotImplementedError

```

```

In [ ]: # Marking scheme for C3 -- do not edit.
with GRADER.check('C3', points=4):
    assert set(runs) == {"baseline", "gepa"} and all(len(v) == 3 for v in runs.values())
    assert all(0.0 <= x <= 1.0 for v in runs.values() for x in v)
    _m = {k: st.mean(v) for k, v in runs.items()}
    _s = {k: st.stdev(v) for k, v in runs.items()}
    _expected = ("significant" if abs(_m["gepa"] - _m["baseline"]) > 2 * max(_s.values())
                 else "not significant")
    assert verdict_c3 == _expected
    assert {"calls", "usd"} <= set(c3_cost)

```

Your answer:

(Write your answer here.)

Part D — Architecture: agent or pipeline, the price of evidence, and a gated skill

Three decisions remain: whether the system is one tool-calling loop or a decomposed pipeline; how much evidence is worth buying; and how the deployed skill is allowed to change itself.

D1. One agent loop, or a pipeline with a verifying critic? (8 pts)

- Write `AGENT_PROMPT` : a system prompt for a tool-calling agent over `A.build_registry_tools(ws)` that must end its reply with one JSON object `{"level": ..., "smells": [...], "decision": ..., "report": ...}`; and `parse_triage_answer(text) -> dspy.Prediction` (fields `level`, `smells`, `decision`, `report`; the *last* JSON object that has a `decision` key, possibly inside a code fence or prose, may contain nested brackets; empty fields when there is none).
- Write `critic(pred, evidence) -> (dspy.Prediction, list[str])`: a deterministic verification stage with the raw evidence JSON (`A.gather_evidence`). It overrides any field the evidence decides — `level`, reportable smells, decision (your A1 functions) — keeps the report, and returns one human-readable string per correction.
- On every `test` submission, run (a) the agent (fresh workspace each; keep the logs in `agent_logs[id]`), (b) the tuned program from C1, (c) the same program followed by the critic. Build `d1` with columns `approach` (`"agent"`, `"program"`, `"program+critic"`), `id`, `score` (`intake_scorer`), `tool_calls`, `usd` (`llm.chat_usage(...)[ "usd_estimate" ]` for the agent, `llm.meter` for the program).

In writing: compare the three on score, cost and legibility. If the critic can override all three structured fields, what is Claude for in this system — and how would you evaluate that part?

Background and design decisions — D1

The architecture question (Background §10). The **agent** lets Claude choose the tool calls; the **program** fixes them; the **critic** checks the program's structured answer against the raw evidence using your A1 functions. Same tools, same scorer: compare them on score, cost and how easy the run is to audit.

Decisions you make:

- **Agent prompt.** What procedure and output contract to give it. A fixed final JSON object makes the answer parseable; stating the policy in the prompt is the agent's version of the *guidelines* program.
- **Parsing.** Agents write prose around their answer; your parser decides how forgiving to be. Taking the *last* object with a `decision` key handles drafts that are corrected later in the same reply.
- **What the critic may override.** Everything the evidence decides (level, smells, decision), but not the report. The written part asks what that leaves Claude to do.
- **Fresh workspace per agent run**, so each `agent_logs[id]` is one trajectory for D2.

Contract — D1

```
AGENT_PROMPT: str
def parse_triage_answer(text: str) -> dspy.Prediction
def critic(pred, evidence: str) -> tuple[dspy.Prediction, list[str]]
agent_logs: dict[str, ToolCallLog]; d1: pd.DataFrame
```

`parse_triage_answer(text)` — `text` is the agent's final reply (`run.answer`): prose, possibly a Markdown code fence, possibly several JSON objects, possibly nested braces. Find the last JSON object that has a **"decision" key** (tip: try `json.JSONDecoder().raw_decode(text, i)` at every `{ }`). Return `dspy.Prediction(level=str, smells=list[str], decision=str, report=str)`, with `smells` through `ev.parse_label_list`. No such object → `level=""`, `smells=[]`, `decision=""`, `report=""`; never raise.

`critic(pred, evidence)` — a deterministic verifier.

parameter	type	meaning
<code>pred</code>	a prediction (may be an empty <code>dspy.Prediction()</code>)	the model's answer; read it with <code>getattr(..., default)</code>
<code>evidence</code>	<code>str</code>	the JSON text from <code>A.gather_evidence(submission_id)</code>

From the evidence take `spectrum.level` (measured), `submission.declared_level` and the set of `scanner_findings[*].smell` ; compute the correct smells and decision with your A1 functions. Return a tuple:

1. a new `dspy.Prediction(level=measured, smells=<reportable list>, decision=<policy decision>, report=<pred's report, or ">`;
2. `list[str]` : one human-readable correction per field the model got wrong (for example `"decision 'revise' -> 'accept'"` , one entry per dropped or added smell). An empty list means the model agreed with the evidence on all three fields.

Running the three approaches on `test`

- agent: a fresh `ws = A.RegistryWorkspace()` per item; `agent, _ = agents.build_agent(ws, system_prompt=AGENT_PROMPT, tools=A.build_registry_tools(ws))` ; `run = agents.run_agent(agent, ws, "Triage the registry submission '<id>'")` ; `score = parse_triage_answer(run.answer)` ; store `agent_logs[id] = ws.log` .
- program: `tuned(submission=id)` inside `llm.meter(lm)` .
- program+critic: `critic(<program prediction>, A.gather_evidence(id))[0]` .

`d1` — 24 rows (3 approaches × 8 test items):

column	type	content
approach	str	"agent" , "program" , "program+critic"
id	str	the submission id
score	float	<code>intake_scorer(ex, prediction).score</code>
tool_calls	int	calls in that item's workspace log
usd	float	agent: <code>llm.chat_usage(run.messages)["usd_estimate"]</code> ; program rows: that call's <code>llm.meter</code> "usd"

```
In [ ]: AGENT_PROMPT = "TODO"

def parse_triage_answer(text: str) -> dspy.Prediction:
    """Last JSON object with a "decision" key -> Prediction(level, smells, decision)
    # TODO
    raise NotImplementedError

def critic(pred, evidence: str) -> tuple[dspy.Prediction, list[str]]:
    """Override level/smells/decision from the evidence; keep the report (see Contr
    # TODO: return (corrected prediction, list of corrections)
    raise NotImplementedError

agent_logs = {}
d1 = None # TODO
```

```
In [ ]: # Marking scheme for D1 -- do not edit.
with GRADER.check('D1', points=4):
    p = parse_triage_answer('Done.\n``json\n{"level": "taxonomy", "smells": ["miss
        "decision": "revise", "report": "Label mr:X."}\n``')
    assert (p.level, p.smells, p.decision) == ("taxonomy", ["missing-label"], "revi
    p = parse_triage_answer('{"note": {"a": 1}} then {"level": "thesaurus", "smells
        "decision": "accept", "report": "x", "extra": {"k": [1
    assert (p.level, p.smells, p.decision) == ("thesaurus", [], "accept")
    p = parse_triage_answer("I think it should be rejected.")
    assert (p.level, p.smells, p.decision) == ("", [], "")

    evidence = A.gather_evidence("timetable-service-thesaurus")
    bad = dspy.Prediction(level="taxonomy", smells=["no-disjointness", "missing-lab
        decision="revise", report="keep me")
    fixed, corrections = critic(bad, evidence)
    assert (fixed.level, list(fixed.smells), fixed.decision, fixed.report) == \
        ("thesaurus", [], "accept", "keep me")
    assert len(corrections) >= 4
    good = dspy.Prediction(level="thesaurus", smells=[], decision="accept", report=
    assert critic(good, evidence)[1] == []

    assert isinstance(d1, pd.DataFrame)
    assert set(d1["approach"]) == {"agent", "program", "program+critic"}
    assert {"approach", "id", "score", "tool_calls", "usd"} <= set(d1.columns)
    assert all(set(g["id"]) == {ex.id for ex in test} for _, g in d1.groupby("appro
    assert set(agent_logs) == {ex.id for ex in test}
    _crit = d1[d1.approach == "program+critic"].set_index("id")["score"]
    assert all(abs(_crit[ex.id] - intake_scorer(ex, critic(dspy.Prediction()), A.gat
        for ex in test)
```

Your answer:

(Write your answer here.)

D2. The price of evidence: an MDP (8 pts)

`mdp.EvidenceMDP(kinds, score, step_cost)` models a reviewer buying evidence (one tool call per kind in `A.EVIDENCE_KINDS`) and then submitting; the submit reward is the answer quality the evidence supports. Define that precisely: a field is **determined** by an evidence set `E` when the policy gives the same answer for every value the missing evidence could take — the measured level ranges over all four levels if `"spectrum"` is not in `E`, the declared level over all four if `"record"` is not in `E`; without `"smells"`, nothing about defects or the decision is determined (an unseen scan could return anything). The level is determined iff `"spectrum"` is in `E`.

1. Implement `determined(row, E) -> dict` (keys `level`, `smells`, `decision`, bools) and `quality(row, E) = 0.3*level + 0.4*smells + 0.3*decision` — the scorer's weights, because a determined field is answered correctly.
2. For every submission, solve its MDP at `step_cost=0.05` with `mdp.value_iteration`; build `d2` with columns `id`, `v_star`, `plan` (the evidence

kinds the greedy optimal rollout buys, as a sorted list).

3. Pool the items (score = mean quality over all 24) — a reviewer who does not know in advance which submission it faces. Sweep `step_cost` over `np.round(np.arange(0.0, 0.31, 0.01), 2)` and set `c_record` to the smallest cost at which the pooled optimal plan no longer buys "record".
4. Replay each D1 agent trajectory (`mdp.episode_from_tool_log` , with its D1 score) in its item's MDP and build `d2_regret` with columns `id` , `agent_return` , `v_star` , `regret` .

In writing: explain `c_record` analytically; say what the per-item V^* assumes that no real reviewer has; and interpret the agent's regret — how much is procedure, how much is answer quality, and how much is the reward model's fault?

Background and design decisions — D2

The MDP (Background §11) prices evidence. State = the evidence kinds held; actions = buy one more kind (one tool call, cost `step_cost`) or *submit*; the submit reward = how good an answer that evidence supports. Solving it gives the best achievable trade-off, V^* , and the agent's regret is its distance from it.

Decisions you make:

- **What "the evidence supports an answer" means.** Here: a field counts as answered only if the policy gives the same answer for **every** value the missing evidence could take. That is a pessimistic, checkable definition; the written part asks what it assumes.
- **Per item vs pooled.** A per-item MDP assumes the reviewer knows in advance which submission it faces (and so which evidence will matter); the pooled MDP does not. The gap between them is the value of that foresight.
- **The reward model.** The step cost is a made-up price; regret is only meaningful if the cost reflects what actually costs money (tokens per turn, not tool calls).

Contract — D2

```
def determined(row: dspy.Example, E: Iterable[str]) -> dict[str, bool]
def quality(row: dspy.Example, E: Iterable[str]) -> float
d2: pd.DataFrame; c_record: float; d2_regret: pd.DataFrame
```

parameter	type	meaning
<code>row</code>	dataset row	fields read: <code>level</code> , <code>declared_level</code> , <code>raw_smells</code>
<code>E</code>	<code>Iterable[str]</code> , a subset of <code>A.EVIDENCE_KINDS</code>	the evidence a reviewer holds, e.g. <code>{"spectrum", "smells"}</code>

`determined(row, E)` returns `{"level": bool, "smells": bool, "decision": bool}` :

- `level` is `True` iff `"spectrum"` in `E` ;
- without `"smells"` in `E` , `smells` and `decision` are both `False` ;
- otherwise let the candidate measured levels be `[row.level]` if `"spectrum"` in `E` , else all of `ont.SPECTRUM_LEVELS` , and the candidate declared levels be `[row.declared_level]` if `"record"` in `E` , else all four. `smells` is `True` iff `reportable_smells(row.raw_smells, m)` is the same for every candidate `m` ; `decision` is `True` iff `intake_decision(...)` is the same for every candidate pair `(m, d)` .

Example: for `signalling-equipment` (it has a subsumption cycle), `determined(row, {"smells"}) == {"level": False, "smells": False, "decision": True}` , because every level gives *reject* but the waiver makes the reported smells depend on the level.

`quality(row, E) = 0.3*level + 0.4*smells + 0.3*decision` (bools count as 0/1). It is called thousands of times; memoise it (e.g. `functools.lru_cache` on `(row.id, frozenset(E))`).

MDP API

call	returns
<code>mdp.EvidenceMDP(A.EVIDENCE_KINDS, score, step_cost=0.05)</code>	an MDP; <code>score</code> is a function <code>frozenset[str] -> float</code> (e.g. <code>lambda E: quality(ex, E)</code>)
<code>mdp.value_iteration(M)</code>	<code>(V, pi) : V[state] -> float , pi[state] -> action</code> ; the start value is <code>V[M.initial_state()]</code>
<code>mdp.run_episode(M, mdp.greedy_policy(pi))</code>	an <code>Episode</code> ; <code>.actions</code> is the list of actions taken, ending with <code>mdp.SUBMIT</code> (<code>"submit"</code>)
<code>mdp.episode_from_tool_log(log, M, A.TOOL_TO_EVIDENCE, final_score)</code>	an <code>Episode</code> replaying a real tool log; <code>.discounted_return()</code> is its return

In the lambda, bind the loop variable (`lambda E, ex=ex: quality(ex, E)`), or every MDP scores the last row.

name	type	content
<code>d2</code>	DataFrame, 24 rows	<code>id</code> , <code>v_star</code> (float), <code>plan</code> (sorted <code>list[str]</code> of the evidence kinds the greedy rollout buys, without <code>"submit"</code>)
<code>c_record</code>	float	smallest <code>step_cost</code> in <code>np.round(np.arange(0.0, 0.31, 0.01), 2)</code> at which the pooled optimal plan (score = mean quality over all 24 rows) no longer buys <code>"record"</code>

name	type	content
d2_regret	DataFrame, 8 rows	id, agent_return, v_star, regret = v_star - agent_return, using agent_logs[id] and that item's D1 agent score

```
In [ ]: import functools
import numpy as np
print(A.EVIDENCE_KINDS)
print(A.TOOL_TO_EVIDENCE)
```

```
In [ ]: def determined(row: dspy.Example, E: Iterable[str]) -> dict[str, bool]:
    """{"level", "smells", "decision"} -> is the field determined by evidence E? (C
    # TODO
    raise NotImplementedError

def quality(row: dspy.Example, E: Iterable[str]) -> float:
    """0.3*level + 0.4*smells + 0.3*decision over determined(row, E) (Contract D2).
    # TODO
    raise NotImplementedError

d2 = None          # TODO (2)
c_record = None    # TODO (3)
d2_regret = None   # TODO (4)
```

```
In [ ]: # Marking scheme for D2 -- do not edit.
with GRADER.check('D2', points=5):
    sig, rost, delay = ROW["signalling-equipment"], ROW["rostering-roles"], ROW["de
    assert determined(sig, {"smells"}) == {"level": False, "smells": False, "decisi
    assert determined(rost, {"spectrum", "smells"})["decision"] is False
    assert determined(rost, {"record", "spectrum", "smells"}) == {"level": True, "s
    assert determined(delay, {"record", "smells"}) == {"level": False, "smells": Tr
    assert determined(ROW["crew-competence"], {"spectrum", "smells"})["decision"] i
    assert abs(quality(sig, {"spectrum", "smells"}) - 1.0) < 1e-9 and quality(sig,
    assert isinstance(d2, pd.DataFrame) and len(d2) == 24 and {"id", "v_star", "pla
    _d2 = d2.set_index("id")
    assert list(_d2.loc["signalling-equipment", "plan"]) == ["smells", "spectrum"]
    assert abs(_d2.loc["signalling-equipment", "v_star"] - 0.90) < 1e-6
    assert list(_d2.loc["rostering-roles", "plan"]) == ["record", "smells", "spectr
    assert abs(_d2.loc["rostering-roles", "v_star"] - 0.85) < 1e-6
    assert abs(c_record - 0.10) <= 0.015
    assert isinstance(d2_regret, pd.DataFrame) and set(d2_regret.id) == {ex.id for
    assert {"agent_return", "v_star", "regret"} <= set(d2_regret.columns)
    assert all(abs(r.regret - (r.v_star - r.agent_return)) < 1e-3 for r in d2_regre
```

Your answer:

(Write your answer here.)

D3. Ship it as a skill, behind a gated self-improvement loop (8 pts)

A skill is a versioned bundle: instruction, tools, dataset, metric. `SelfImprovingSkill` serves requests, mines labelled failures, re-optimises on them, and **promotes a candidate only if it beats the current version by `min_gain` on a holdout the optimiser never sees**.

1. Build `skill = Skill(...)` for the triage program (baseline instruction, your scorer, `dataset=test`, the tools the program uses).
2. Build `sis = SelfImprovingSkill(skill, holdout=dev, optimise=..., min_gain=MIN_GAIN)` where `optimise` runs GEPA with `max_metric_calls=SI_BUDGET`. Inside `llm.meter(lm, reflect)` as `d3_cost`:
`serve train ( sis.run_all )`, run one improvement round, then replace `sis.optimise` with a **no-op control** (`lambda program, trainset: program`) and run a second round; finally `skill.evaluate()` so the card's claim is measured on `test`.
3. Save the skill to `config.artifacts_dir() / "ch01_registry_triage_skill.json"` as `skill_path`; print `sis.report()` and `skill.card()`.

In writing: justify your `MIN_GAIN` from C3. What should the control round do, and what would it mean if it were promoted? Why must the holdout differ from the experience the failures were mined from — and why is `test` still not the right holdout here?

Background and design decisions — D3

Packaging and change control (Background §12). A `Skill` keeps the instruction, the tools it assumes, its evaluation data and its metric together and versioned. `SelfImprovingSkill` serves requests, mines the labelled failures, re-optimises, and asks a **gate** whether the candidate is really better.

Decisions you make:

- **MIN_GAIN**. The margin a candidate must beat on the holdout. Set it below the run-to-run noise from C3 and noise gets promoted; set it far above and nothing ever is. Derive it from your measurement.
- **The no-op control round**. It deliberately proposes "no change". A correct gate must reject it; if it is promoted, the gate is broken.
- **Which data is which**. Experience (`train`) is mined for failures; the holdout (`dev`) is the only data the gate reads; the skill card reports on `test`. Mixing any two leaks the answer into the measurement.

Contract — D3

`Skill` (from `oe_course.skills`)

```

Skill(name: str, description: str,
      build: Callable[[str], dspy.Module],      # instruction -> fresh
      program: pass TriageProgram
      scorer,                                   # intake_scorer
      dataset: list,                           # test: what the skill card
      reports on
      instruction: str,                         # BASELINE_INSTRUCTION
      tools: list[str])                        # names of the registry
tools the program uses
Useful members: .version (int, starts at 1), .instruction, .history
( list[SkillVersion] with .score, .dataset, .note ), .evaluate() (scores
  .dataset and records it on the current version), .card() (str), .save(path) -> Path
(JSON).

```

SelfImprovingSkill (from `oe_course.selfimprove`)

```

SelfImprovingSkill(skill, holdout: list, optimise: Callable[[dspy.Module,
list], dspy.Module],
                    *, min_gain: float)

```

- `optimise(program, trainset)` returns a new program. Here, GEPA with `gepa_metric`, `max_metric_calls=SI_BUDGET` and `reflection_lm=reflect`. Replace it later with the no-op control `lambda program, trainset: program` by assigning `sis.optimise = ...`.
- `sis.run_all(examples)` serves each example and stores it in `sis.experience`. Call it **once**, on `train` (the check expects `len(sis.experience) == len(train)`).
- `sis.improve()` runs one round and returns an `ImprovementRound` (`.before`, `.after`, `.promoted`, `.reason`); it promotes only if `after - before >= min_gain` on `holdout`. All rounds are in `sis.rounds`; `sis.report()` summarises them.

name	type	content
<code>MIN_GAIN</code> , <code>SI_BUDGET</code>	<code>float</code> , <code>int</code>	given (0.05, 30); justify <code>MIN_GAIN</code> in writing
<code>skill</code>	<code>Skill</code>	as above
<code>sis</code>	<code>SelfImprovingSkill</code>	<code>holdout=dev</code>
<code>d3_cost</code>	<code>dict</code>	<code>llm.meter(lm, reflect)</code> around serving, both rounds and <code>skill.evaluate()</code>
<code>skill_path</code>	<code>Path</code>	<code>skill.save(config.artifacts_dir() / "ch01_registry_triage_skill.json")</code>

```

In [ ]: MIN_GAIN, SI_BUDGET = 0.05, 30
skill = None      # TODO
sis = None        # TODO
# TODO: serve, improve, control round, evaluate (inside llm.meter -> d3_cost); save

```

```
raise NotImplementedError
```

```
In [ ]: # Marking scheme for D3 -- do not edit.
with GRADER.check('D3', points=3):
    assert isinstance(skill, Skill) and isinstance(sis, SelfImprovingSkill)
    assert len(sis.rounds) == 2 and len(sis.experience) == len(train)
    for r in sis.rounds:
        assert r.promoted == (r.after - r.before >= sis.min_gain), r
    assert skill.version == 1 + sum(r.promoted for r in sis.rounds)
    saved = json.loads(skill_path.read_text(encoding="utf-8"))
    assert len(saved["history"]) == skill.version and saved["instruction"] == skill
    assert skill.history[-1].score is not None and skill.history[-1].dataset == f"{
    assert {"calls", "usd"} <= set(d3_cost)
```

Your answer:

(Write your answer here.)

Grading rubric

Part	Problem	Auto	Manual	Pts
A	A1 The intake policy as code	6	4	10
A	A2 Tools, trajectories, and what a mega-tool hides	4	4	8
B	B1 A diagnostic triage scorer	10	2	12
B	B2 The Claude triage program, and its baseline	4	6	10
B	B3 Validate the LLM judge before trusting it	5	5	10
C	C1 A budgeted GEPA run with a held-out report	5	5	10
C	C2 Was the optimiser worth it — and was the feedback?	3	5	8
C	C3 Is the difference bigger than the noise?	4	4	8
D	D1 One agent loop, or a pipeline with a verifying critic?	4	4	8
D	D2 The price of evidence: an MDP	5	3	8
D	D3 Ship it as a skill, behind a gated self-improvement loop	3	5	8
Total				100

Manual points are for the written answers and for the quality of the engineering evidence (splits, sample sizes, cost, honest negative results).

Self-test

Re-run this cell after finishing. It lists every auto-graded problem and whether its check passed.

```
In [ ]: AUTO_GRADED = ['A1', 'A2', 'B1', 'B2', 'B3', 'C1', 'C2', 'C3', 'D1', 'D2', 'D3']  
_ = GRADER.summary(AUTO_GRADED)
```